詳細設計書

Loggit

Version 1.0　　作成日：2026 年 2 月 23 日

A. 打刻（activity_logs）まわり

A-1. 計測開始/停止のトランザクション設計

計測開始時のフローは以下の通り。競合を防ぐために悲観的ロック（SELECT FOR UPDATE）を使用する。

```
DB::transaction(function () use ($user) {
    // 1. 対象ユーザーの行をロック
    $active = ActivityLog::where('user_id', $user->id)
                          ->whereNull('ended_at')
                          ->lockForUpdate()
                          ->first();
    // 2. 計測中ログが存在すれば 422 を返す
    if ($active) {
        throw new AlreadyTrackingException();
    }
    // 3. 新規ログを作成（started_at はサーバー時刻）
    return ActivityLog::create([
        'user_id'    => $user->id,
        'started_at' => now(),
    ]);
});
```

※ 停止時も同様に lockForUpdate() を使い、ended_at が NULL であることを確認してから update する

A-2. 複数端末同時 start のレース条件対策

- DB トランザクション＋SELECT FOR UPDATE で基本的な競合を防止
- 現時点は API バリデーションで担保。DB のユニーク部分インデックスは将来検討

※ 将来検討：WHERE ended_at IS NULL の部分インデックスによる DB 制約追加

A-3. started_at / ended_at のサーバー時刻基準

- クライアントから時刻を受け取らない。started_at・ended_at は全てサーバー側で now()をセット
- API のリクエストボディに時刻フィールドを含めない設計とする
- 手動編集（note 等）は許可するが、時刻の手動上書きは不可

A-4. 24 時間自動停止の実装方式

Laravel のスケジューラ（cron）を使い、毎時 0 分に以下のバッチを実行する。

```
// app/Console/Commands/AutoStopActivityLogs.php
ActivityLog::whereNull('ended_at')
    ->where('started_at', '<=', now()->subHours(24))
    ->update([
        'ended_at'     => DB::raw('DATE_ADD(started_at, INTERVAL 24 HOUR)'),
        'auto_stopped' => true,
    ]);
```

```
// app/Console/Kernel.php
$schedule->command('activitylogs:auto-stop')->hourly();
```

※ ended_at はstarted_at + 24 時間をセット（サーバー時刻基準）

A-5. 自動停止の通知実装

- activity_logs テーブルに auto_stopped BOOLEAN NOT NULL DEFAULT FALSE を追加
- ダッシュボード表示時に auto_stopped=true かつ未確認のログを API で検出して通知バナーを表示
- ユーザーが通知を確認したら acknowledged_at TIMESTAMP NULL を更新して非表示にする

追加カラム	型	制約	説明
auto_stopped	BOOLEAN	DEFAULT false	自動停止フラグ
acknowledged_at	TIMESTAMP	NULL	通知確認日時

A-6. 重複停止の 422 レスポンス

```
// 停止済みログへの再停止リクエスト時
{
    "message": "この活動ログはすでに停止されています",
    "errors": {}
}
```

A-7. 計測中ログの note 編集

- 計測中（ended_at=NULL）のログに対して note の編集を許可する

- PUT /api/activity-logs/{id} で note のみ更新可能
- started_at・ended_at の更新は API で拒否する（400 を返す）

A-8. 日跨ぎログの集計注記

- ダッシュボードの週次・月次集計値の下に「※ 開始日基準で集計しています」と小テキストで表示
- 例：23:00 開始→翌 2:00 終了の 3 時間ログは開始日の活動としてカウント

B. 時間配賦（task_time_logs）まわり

B-9. 配賦 UI の入力仕様

- 停止後に配賦モーダルを自動表示（スキップ可能）
- モーダル上部に「総活動時間：XX 分」を表示
- タスク選択（プロジェクト→タスクの 2 段階セレクト）＋分単位の数値入力
- 「＋ タスクを追加」ボタンで行を増やして複数タスクに配賦可能
- モーダル下部にリアルタイムで「配賦合計：XX 分 / 総活動時間：XX 分」を表示
- 合計が総活動時間を超えた場合は赤色テキスト＋警告メッセージを表示（保存は可能）

B-10. 端数処理の具体仕様

- 入力フィールドは整数のみ受け付ける（分単位の整数入力）
- API の保存時に floor 処理（切り捨て）を実施
- 集計表示時も分単位の整数のまま扱い、表示時のみ「1h 30m」形式に変換

```
// 表示変換例
function formatMinutes(minutes) {
    const h = Math.floor(minutes / 60);
    const m = minutes % 60;
    if (h === 0) return `${m}m`;
    if (m === 0) return `${h}h`;
    return `${h}h ${m}m`;
}
```

B-11. 配賦超過の警告仕様

- 配賦合計 > 総活動時間になった時点でリアルタイムに赤色警告を表示
- 警告文：「配賦時間が総活動時間を超えています（超過：XX 分）」
- 保存ボタンは無効化しない。警告を確認の上で保存可能
- タスク詳細パネルの累計時間も同様に赤色で表示

B-12. 配賦の編集・再配賦フロー

- 停止直後のモーダルで初回配賦
- 活動ログ一覧の各ログ行に「配賦する / 編集」ボタンを配置し、後からいつでも再編集可能
- タスク詳細パネルからも手動で配賦の追加・削除が可能

B-13. activity_log_id=NULL（手動入力）のユースケース

- タスク詳細サイドパネルの「時間を手動追加」ボタンから作成
- 活動ログとの紐付けなしで直接 duration_minutes を入力する
- 「過去の作業を思い出して記録したい」「ログを取り忘れた」ケースに対応

B-14. 手動入力配賦の日時管理

task_time_logs テーブルに worked_on DATE NULL カラムを追加する。

追加カラム	型	制約	説明
worked_on	DATE	NULL	作業日（手動入力時に任意指定）

- activity_log 経由の配賦：worked_on = started_at の日付を自動セット
- 手動入力の配賦：worked_on はユーザーが任意で入力（NULL 許容）

B-15. 計測中ログへの配賦禁止の実装

- API バリデーション：配賦対象の activity_log の ended_at が NULL の場合は 422 を返す
- フロント：計測中のログには配賦ボタンを非活性（disabled）にする
- エラーメッセージ：「計測中の活動ログには配賦できません」

C. 整合性（project / task / log）

C-16. プロジェクト別時間の集計軸

- 「合計活動時間」→ activity_logs ベース（ended_at - started_at で計算）
- 「タスク別配賦時間」→ task_time_logs ベース（duration_minutes の合計）
- 「プロジェクト別配賦時間」→ task_time_logs.task_id → tasks.project_id で集計

※ activity_logs.project_id は絞り込みフィルターとしての用途に留める

C-17. activity_logs.project_id の役割定義

- 「その活動をどのプロジェクトのために行ったか」の補助情報として位置づける
- 集計の主軸は task_time_logs に寄せる
- 活動ログ一覧画面でプロジェクットによる絞り込みフィルターとして使用
- 打刻開始時に project_id を任意で指定できる UI を提供する

C-18. タスク・プロジェクト削除時の集計影響

- tasks はソフトデリート（deleted_at カラム追加）
- 削除済みタスクに紐づく task_time_logs は保持。集計には引き続き含める
- 活動ログ一覧では削除済みタスク名を「削除済みタスク」とグレーテキストで表示＋復元ボタン
- projects は物理削除。配下の tasks も CASCADE DELETE で物理削除
- project 削除時の activity_logs.project_id は SET NULL で保持

削除種別	task_time_logs の扱い
タスクをソフトデリート	task_time_logs は保持。集計に含める。表示は「削除済みタスク」
プロジェクトを物理削除	tasks→task_time_logs も CASCADE 削除。集計から消える

C-19. サブタスク階層制限の実装

API で tasks テーブルを parent_task_id で再帰的に辿り、depth（階層数）を計算する。

```
// TaskService.php
public function getDepth(Task $task): int {
    $depth = 1;
    $current = $task;
    while ($current->parent_task_id !== null) {
        $current = Task::find($current->parent_task_id);
        $depth++;
        if ($depth > 3) break; // 無限ループ防止
    }
    return $depth;
}
```

- サブタスク作成時：親タスクの depth が 3 以上の場合は 422
- 循環参照防止：parent_task_id に自分自身の id をセットしようとした場合は 422
- エラーメッセージ：「サブタスクは 3 階層までしか作成できません」

D. API 詳細（入出力仕様）

D-20. Request / Response スキーマ

Laravel の API Resource クラスでレスポンスを統一する。

認証

エンドポイント	フィールド	型	必須/任意
POST /api/register	name	string	必須
	email	string	必須・UNIQUE
	password	string	必須・min:8
POST /api/login	email	string	必須
	password	string	必須

プロジェクト

エンドポイント	フィールド	型	必須/任意
POST /api/projects	name	string	必須・max:255
	description	string	任意
	color	string	任意・HEX 形式
	due_date	date	任意・Y-m-d 形式
PUT /api/projects/{id}	name	string	任意・max:255
	description	string	任意
	color	string	任意・HEX 形式
	due_date	date	任意・Y-m-d 形式

タスク

エンドポイント	フィールド	型	必須/任意
POST /api/projects/{id}/tasks	title	string	必須・max:255
	description	string	任意
	status_id	integer	必須
	priority	integer	任意・1/2/3
	due_date	date	任意・Y-m-d 形式

エンドポイント	フィールド	型	必須/任意
	parent_task_id	integer	任意（サブタスク時）

活動ログ

エンドポイント	フィールド	型	必須/任意
POST /api/activity-logs/start	project_id	integer	任意
	note	string	任意
PUT /api/activity-logs/{id}	note	string	任意
	project_id	integer	任意
POST /api/tasks/{id}/time-logs	duration_minutes	integer	必須・min:1
	activity_log_id	integer	任意（NULL=手動）
	worked_on	date	任意・手動時のみ
	note	string	任意

D-21. バリデーション 400/422 の使い分け

ステータス	使用ケース
400 Bad Request	リクエスト形式の問題（必須パラメータ欠落・型の不一致・形式エラー）
422 Unprocessable	ビジネスルール違反（同時計測中・計測中への配賦・階層超過・ステータス件数超過等）
403 Forbidden	他ユーザーのリソースへのアクセス（Policy で NG 判定）
404 Not Found	存在しないリソースへのアクセス

エラーレスポンスボディの統一形式:

```
{
    "message": "エラー内容の説明（日本語）",
    "errors": {
        "field_name": ["バリデーション詳細"]  // バリデーション違反時のみ
    }
}
```

D-22. ページネーション仕様

Laravel の paginate メソッドをベースにした JSON レスポンス形式で統一する。

```
{
    "data": [ ...リソース配列... ],
    "meta": {
        "current_page": 1,
        "per_page": 20,
        "total": 100,
        "last_page": 5
    },
    "links": {
        "prev": null,
        "next": "/api/activity-logs?page=2"
    }
}
```

※ per_page のデフォルトは20 件。クエリパラメータ per_page で変更可能（max:100）

D-23. ソート/フィルターのクエリ仕様

タスク横断一覧 GET /api/tasks

パラメータ	説明
sort=created_at_desc（デフォルト）	作成日降順（新しい順）
sort=due_date_asc	期限昇順（近い順）
sort=priority_desc	優先度降順（高い順）
project_id=1	プロジェクトで絞り込み
status_id=2	ステータスで絞り込み
priority=3	優先度で絞り込み（1/2/3）
due_date=2026-03-01	指定日以前の期限で絞り込み

活動ログ一覧 GET /api/activity-logs

パラメータ	説明
sort=started_at_desc（デフォルト）	開始日時降順
project_id=1	プロジェクトで絞り込み
period=week	今週分のみ（日曜始まり）
period=month	今月分のみ
date_from=2026-02-01	指定日以降で絞り込み（started_at 基準）

パラメータ	説明
date_to=2026-02-28	指定日以前で絞り込み（started_at 基準）

D-24. 権限チェックの実装方式

Laravel の Policy クラスでリソースごとに権限を定義する。Controller ではなく Policy に権限ロジックを集約する。

Policy クラス	チェック内容
ProjectPolicy	projects.user_id === 認証ユーザーの id
TaskPolicy	tasks.project_id → projects.user_id === 認証ユーザーの id
ActivityLogPolicy	activity_logs.user_id === 認証ユーザーの id
TaskTimeLogPolicy	task_time_logs.task_id → tasks → projects.user_id === 認証ユーザーの id

※ 権限チェックNG の場合は403 Forbidden を返す

D-25. 例外・業務ルール違反のメッセージ規約

- message フィールドは日本語で統一
- 文言パターン：「〇〇はすでに△△されています」「〇〇は△△できません」
- errors フィールドはバリデーション違反時のみ使用。業務ルール違反時は空オブジェクト{}

ケース	message
同時計測中ログあり	すでに計測中の活動ログがあります
重複停止	この活動ログはすでに停止されています
計測中への配賦	計測中の活動ログには配賦できません
階層超過	サブタスクは３階層までしか作成できません
ステータス追加超過	ステータスはこれ以上追加できません
ステータス削除制限	ステータスは最低１件必要です
循環参照	循環参照となるサブタスクは作成できません

E. 集計（週次・月次ダッシュボード）

E-26. 集計方式

- 「合計活動時間」→ activity_logs ベース（TIMESTAMPDIFF(MINUTE, started_at, ended_at)の合計）
- 「タスク別・プロジェクト別配賦時間」→ task_time_logs ベース（duration_minutes の合計）
- ended_at=NULL の計測中ログは集計から除外する

E-27. 週の開始日曜固定の期間算出

Laravel の Carbon を使って日曜始まりの週を算出する。

```
use Carbon\Carbon;

// 週次
$start = Carbon::now('Asia/Tokyo')->startOfWeek(Carbon::SUNDAY)->utc();
$end   = Carbon::now('Asia/Tokyo')->endOfWeek(Carbon::SATURDAY)->utc();

// 月次
$start = Carbon::now('Asia/Tokyo')->startOfMonth()->utc();
$end   = Carbon::now('Asia/Tokyo')->endOfMonth()->utc();
```

E-28. 日跨ぎログの開始日基準の集計実装

集計クエリの WHERE 条件は started_at のみで絞り込む。ended_at は条件に含めない。

```
ActivityLog::where('user_id', $userId)
    ->whereNotNull('ended_at')
    ->whereBetween('started_at', [$start, $end])
    ->selectRaw('SUM(TIMESTAMPDIFF(MINUTE, started_at, ended_at)) as
total_minutes')
    ->value('total_minutes') ?? 0;
```

E-29. 集計の丸め規則

- DB 保存は分単位の整数（INT）
- 表示変換：60 分以上は「Xh Ym」形式、60 分未満は「Xm」のみ
- 小数点は使わない
- 四捨五入なし・切り捨て統一

分数	表示
45 分	45m
60 分	1h
91 分	1h 31m

分数	表示
120 分	2h

E-30. キャッシュ戦略

- 現時点ではキャッシュなし。インデックス設計で対応
- ダッシュボードの集計クエリが重くなった場合は Laravel Cache（Redis）の導入を検討
- キャッシュキー設計（拡張時）： dashboard:user:{user_id}:week / dashboard:user:{user_id}:month
- 失効条件（拡張時）： activity_log・task_time_log の作成・更新・削除時にキャッシュをクリア

F. DB 実装（マイグレーション / 制約）

F-31. FK（CASCADE / SET NULL）のマイグレーション実装

```
// CASCADE 例（projects 削除時に tasks も削除）
$table->foreignId('project_id')
      ->constrained()
      ->cascadeOnDelete();

// SET NULL 例（projects 削除時に activity_logs.project_id を NULL）
$table->foreignId('project_id')
      ->nullable()
      ->constrained()
      ->nullOnDelete();

// SET NULL 例（activity_logs 削除時に task_time_logs.activity_log_id を NULL）
$table->foreignId('activity_log_id')
      ->nullable()
      ->constrained('activity_logs')
      ->nullOnDelete();
```

F-32. CHECK 制約の MySQL 対応方針

- MySQL 8.0.16 以上を前提として実装する
- started_at < ended_at の CHECK 制約は DB レベルで追加（保険として）
- API バリデーションで確実に担保しているため、CHECK 制約は二重の保護として位置づける

```
// マイグレーション内で CHECK 制約を追加
DB::statement(
    'ALTER TABLE activity_logs
     ADD CONSTRAINT chk_ended_after_started
     CHECK (ended_at IS NULL OR ended_at > started_at)'
);
```

F-33. インデックスの追加方針

- 基本設計で定義済みのインデックスをマイグレーションで実装
- 実装が進んだ段階で EXPLAIN を使って実際のクエリに合わせて調整
- 複合インデックスは必要に応じて追加（例：activity_logs.user_id + started_at）

```
// マイグレーション例
$table->index('user_id');
$table->index('started_at');
$table->index(['user_id', 'started_at']); // 複合インデックス
```

F-34. ステータス 3 件固定・削除制限の実装

API バリデーションを StatusService クラスに集約して実装する。

```
// StatusService.php
public function canAdd(Project $project): bool {
    return $project->statuses()->count() < 3;
}

public function canDelete(Project $project): bool {
    return $project->statuses()->count() > 1;
}
```

操作	エラーメッセージ
ステータスが 3 件の状態で追加	ステータスはこれ以上追加できません
ステータスが 1 件の状態で削除	ステータスは最低 1 件必要です

G. フロント仕様（画面遷移・状態）

G-35. 配賦モーダルの表示条件

- 計測停止直後に自動で配賦モーダルを表示
- 「スキップ」ボタンで閉じた場合、活動ログ一覧の該当ログ行に「配賦する」ボタンを表示
- 配賦済みのログにも「編集」ボタンを表示し、後からいつでも再編集可能
- 計測中のログには配賦ボタンを非活性（disabled）にする

G-36. タスク詳細パネルの累計時間・超過警告

- タスク詳細パネルを開いたタイミングで GET /api/tasks/{id}/time-logs を呼び出し累計時間を取得
- 配賦モーダルで入力中はフロント側でリアルタイム計算（API コールなし）
- 累計配賦時間 > 総活動時間になった時点で赤色テキスト＋警告メッセージに切り替え
- 保存後は再度 API を呼び出して最新の累計時間を反映

G-37. タスク横断一覧の UI 仕様

- フィルター・ソートの状態は URL クエリパラメータに同期
- 例：/tasks?sort=due_date_asc&project_id=1&status_id=2
- ページリロード後も URL から状態を復元する
- フィルター UI はサイドパネルまたはヘッダー部分にドロップダウンで配置
- フィルター変更のたびに API を再取得（デバウンス：300ms）

G-38. ローディング/空状態/エラー表示の統一ルール

状態	表示仕様
ローディング	スケルトン UI で統一（スピナーは使わない）
空状態	「まだ〇〇がありません」＋作成ボタンを表示
エラー（API 失敗）	トースト通知で画面右下に表示・3 秒で自動消去・赤色
成功（作成・更新）	トースト通知で画面右下に表示・3 秒で自動消去・緑色
権限エラー（403）	「アクセス権限がありません」トースト表示
未認証（401）	ログイン画面へリダイレクト

H. Laravel 実装設計

H-1. ディレクトリ構成

- app/Http/Controllers/API/ → 各リソースの API コントローラー
- app/Http/Requests/ → FormRequest バリデーションクラス
- app/Http/Resources/ → API リソースクラス（レスポンス整形）
- app/Models/ → Eloquent モデル
- app/Services/ → ビジネスロジック（TaskService, ActivityLogService, StatusService 等）
- app/Policies/ → 権限チェック（ProjectPolicy, TaskPolicy 等）
- app/Exceptions/ → 業務例外クラス（AlreadyTrackingException 等）
- app/Console/Commands/ → バッチコマンド（AutoStopActivityLogs 等）

H-2. Eloquent リレーション設計

モデル	リレーション
User	hasMany: Project, ActivityLog, UserSetting
Project	belongsTo: User / hasMany: Task, ProjectStatus, ActivityLog

モデル	リレーション
ProjectStatus	belongsTo: Project / hasMany: Task
Task	belongsTo: Project, ProjectStatus / hasMany: TaskTimeLog / belongsTo: Task(parent) / hasMany: Task(children) / SoftDeletes
ActivityLog	belongsTo: User, Project / hasMany: TaskTimeLog
TaskTimeLog	belongsTo: Task, ActivityLog

H-3. シーケンス：計測開始〜停止〜配賦

【計測開始】

- 1. POST /api/activity-logs/start
- 2. ActivityLogPolicy で認証チェック
- 3. StartActivityLogRequest で project_id・note をバリデーション
- 4. ActivityLogService::start() でトランザクション開始
- 5. SELECT FOR UPDATE で計測中ログの存在確認
- 6. 存在すれば AlreadyTrackingException → 422
- 7. ActivityLog::create(['started_at' => now(), ...])
- 8. ActivityLogResource でレスポンス返却

【計測停止】

- 1. POST /api/activity-logs/{id}/stop
- 2. ActivityLogPolicy で所有権チェック
- 3. ActivityLogService::stop() でトランザクション開始
- 4. SELECT FOR UPDATE でログ取得
- 5. ended_at が既にセットされていれば AlreadyStoppedException → 422
- 6. ended_at = now() で update、auto_stopped = false を明示セット
- 7. 配賦モーダル表示用に総活動時間も合わせてレスポンス

【配賦】

- 1. POST /api/tasks/{id}/time-logs
- 2. TaskTimeLogPolicy で所有権チェック
- 3. StoreTaskTimeLogRequest でバリデーション
- 4. activity_log_id が指定された場合、ended_at=NULL なら 422
- 5. TaskTimeLog::create([...]) で保存
- 6. worked_on：activity_log 経由なら started_at の日付を自動セット

I. 推奨実装順序

フェーズ	内容	詳細
1	環境構築	Laravel / React / MySQL セットアップ・Sanctum 導入

フェーズ	内容	詳細
2	DB 構築	全テーブルのマイグレーション・FK・インデックス・CHECK 制約
3	認証	登録・ログイン・ログアウト・Policy 基盤
4	プロジェクト管理	CRUD・ステータスカスタマイズ・3 件固定バリデーション
5	タスク管理	CRUD・ソフトデリート・サブタスク階層・priority TINYINT
6	活動ログ	打刻・停止・トランザクション・24 時間自動停止バッチ
7	時間配賦	配賦モーダル・手動入力・超過警告
8	集計・ダッシュボード	週次・月次集計・Carbon 期間算出
9	フロント仕上げ	スケルトン UI・トースト・URL クエリ同期・設定画面
拡張	ガントチャート等	余裕があれば追加実装